



DATA STRUCTURE

Assignment_2

Prepared by:

Name of Student : Chaitanya Dalvi

Roll No: 19

Batch: 2023-27

Dept. of CSE

Que1. What do you mean by tree traversal technique?

Que 2. Differentiate between inorder, preorder and postorder traversal?

Que 3. Write a program to implement Binary Search Tree algorithm?
Explain with the help of an example?

Que 4. How to perform BST Deletion? Explain with the help of an example?

Que 5. What are Balance Tree in Advanced Data Structure?

Que 6. What are AVL Trees and its balancing factor? Draw a tree and show how to balance it?

Que 7. What are the rotations in AVL Tree?

Que 8. What are the properties of Red Black Tree?

Que 9. What are the cases for insertion in Red Black Tree?

Que 10. What are Complete Binary Tree?

Que 11. Differentiate Between:

Binary Tree, Binary Search Tree, Complete Binary Tree

Que 12. What are tree manipulation techniques?

1. What do you mean by tree traversal technique?

Ans:-

Tree traversal refers to the process of visiting every node in a tree data structure exactly once to either read, update, or process its data. There are several techniques for traversing a tree, and they define the order in which nodes are visited. The most common tree traversal methods are **inorder**, **preorder**, and **postorder** for depth-first traversal, and **level-order** traversal for breadth-first traversal.

2. Differentiate between inorder, preorder and postorder traversal?

Ans:-

Inorder Traversal: Visit the **left subtree**, then the **root node**, followed by the **right subtree**. For a binary search tree (BST), this traversal visits nodes in ascending order.

- Example: Left → Root → Right

Preorder Traversal: Visit the **root node** first, then the **left subtree**, and finally the **right subtree**. This traversal is useful for creating a copy of the tree.

- Example: Root → Left → Right

Postorder Traversal: Visit the **left subtree** first, then the **right subtree**, and finally the **root node**. This traversal is useful for deleting the tree.

- Example: Left → Right → Root

3. Write a program to implement Binary Search Tree algorithm? Explain with the help of an example?

Ans:-

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = right = nullptr;
    }
};
```

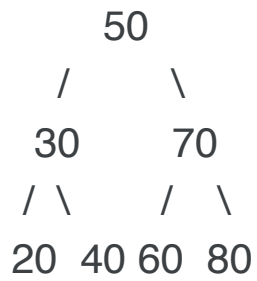
```
Node* insert(Node* root, int value) {
    if (root == nullptr) {
        return new Node(value);
    }
    if (value < root->data) {
        root->left = insert(root->left, value);
    } else if (value > root->data) {
        root->right = insert(root->right, value);
    }
    return root;
}
```

```
void inorder(Node* root) {  
    if (root != nullptr) {  
        inorder(root->left);  
        cout << root->data << " ";  
        inorder(root->right);  
    }  
}
```

```
int main() {  
    Node* root = nullptr;  
    root = insert(root, 50);  
    insert(root, 30);  
    insert(root, 20);  
    insert(root, 40);  
    insert(root, 70);  
    insert(root, 60);  
    insert(root, 80);  
  
    cout << "Inorder traversal: ";  
    inorder(root);  
  
    return 0;  
}
```

Example:

Inserting 50, 30, 20, 40, 70, 60, 80 will result in the following BST structure:



4. How to perform BST Deletion? Explain with the help of an example?

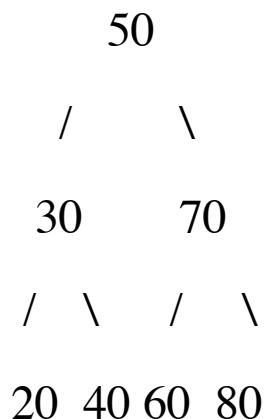
Ans:-

Deletion in a BST can be done in three cases:

- 1. Node has no children (Leaf Node):** Simply remove the node.
- 2. Node has one child:** Remove the node and connect its child to its parent.
- 3. Node has two children:** Replace the node with its inorder successor (the smallest node in its right subtree) or its inorder predecessor (the largest node in its left subtree).

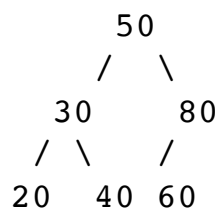
Example:

For the following tree:



If we delete 70:

- 70 has two children, so we replace it with its inorder successor (80).
- The resulting tree becomes:



5. What are Balance Tree in Advanced Data Structure?

Ans:-

A **Balanced Tree** is a tree where the height difference between the left and right subtrees of any node is minimal. This ensures efficient operations like search, insertion, and deletion, typically in logarithmic time. Examples of balanced trees include:

- **AVL Trees**
- **Red-Black Trees**
- **B-Trees**
- **Splay Trees**

6.What are AVL Trees and its balancing factor? Draw a tree and show how to balance it?

Ans:-

An **AVL Tree** is a self-balancing binary search tree where the difference in heights (balancing factor) between the left and right subtrees of any node is at most 1. The balancing factor is defined as:

Balancing Factor = Height of Left Subtree - Height of Right Subtree

When the balance factor is not within $[-1, 1]$, rotations are performed to restore balance.

Example of an Unbalanced AVL Tree:

```
  30
 /
20
/
10
```

This tree has a balance factor of 2 at the root (30), making it unbalanced. To balance it, we perform a **right rotation** at node 30:

```
  20
 /  \
10   30
```

7. What are the rotations in AVL Tree?

Ans:-

Rotations are used to maintain balance in an AVL tree:

1. **Left Rotation:** Used when the right subtree is taller.
2. **Right Rotation:** Used when the left subtree is taller.
3. **Left-Right Rotation:** A left rotation followed by a right rotation.
4. **Right-Left Rotation:** A right rotation followed by a left rotation.

8. What are the properties of Red Black Tree?

Ans:-

A **Red-Black Tree** is a balanced binary search tree with the following properties:

1. Each node is either **red** or **black**.
2. The root is always **black**.
3. **Red** nodes cannot have **red** children (no two consecutive red nodes).
4. Every path from a node to its leaf must have the same number of black nodes.
5. New insertions are always **red**.

9. What are the cases for insertion in Red Black Tree?

Ans:-

When inserting a node in a Red-Black Tree, these cases arise:

1. **Case 1:** The new node is the root; it is colored black.
2. **Case 2:** The parent is black; the tree remains valid.
3. **Case 3:** The parent and the uncle are red; recoloring occurs.
4. **Case 4:** The parent is red, and the uncle is black; rotations and recoloring may be required to maintain balance.

10. What are Complete Binary Tree?

Ans:-

A **Complete Binary Tree** is a binary tree in which all levels are completely filled except possibly the last level, which is filled from left to right. This ensures that the tree is compact, reducing the height and making operations more efficient.

11. Differentiate Between:

Binary Tree, Binary Search Tree, Complete Binary Tree

Ans:-

- **Binary Tree:** A general tree where each node can have at most two children.
- **Binary Search Tree (BST):** A binary tree where each node's left child has a value less than the node, and the right child has a value greater than the node.
- **Complete Binary Tree:** A binary tree in which all levels are fully filled except possibly the last level, which is filled from left to right.

12. What are tree manipulation techniques?

Ans:-

Tree manipulation techniques include:

1. **Insertion:** Adding nodes to the tree.
2. **Deletion:** Removing nodes from the tree.
3. **Traversal:** Accessing nodes in a specific order (inorder, preorder, postorder, level-order).
4. **Balancing:** Ensuring the tree remains balanced (e.g., in AVL or Red-Black Trees) using rotations and reordering.
5. **Merging:** Combining two trees into one.
6. **Splitting:** Dividing a tree into two subtrees based on a certain criterion.